

Lab of Algorithm

Assignment 5

Greedy Algorithms and Activity Scheduling Strategy Analysis

Student ID: 411321344

Name: Wilkend AUGUSTE

Course: Lab of Algorithm

Date: May 5, 2026



National Dong Hwa University

Department of Computer Science and Information Engineering

1. Problem Understanding

The activity selection problem is a classic scheduling problem: given a set of activities, each with a fixed start time and finish time, we want to select the maximum number of activities that can be scheduled without any two of them overlapping. Two activities overlap if one starts before the other finishes.

This problem naturally fits a greedy approach; at each step we make the locally best choice and hope that it leads to the globally optimal solution. The key insight is choosing which "locally best" criterion to use. Not every seemingly reasonable criterion actually works, and a big part of this assignment is understanding why.

The formal definition: given activities $a_1, a_2, \dots, a_n$ where activity a_i has start time s_i and finish time f_i , select a maximum subset S such that for any two activities a_i, a_j in S , either $f_i \leq s_j$ or $f_j \leq s_i$ (they do not overlap).

2. Algorithm Design

2.1 The Correct Greedy Strategy

The optimal strategy is to always pick the activity that finishes earliest among those compatible with the current selection. This is called the Earliest Finish Time First (EFF) strategy. The intuition is simple: by finishing as early as possible, we leave the most room for future activities.

Pseudocode:

```
sort activities by finish time ascending
selected = empty list
lastFinish = -infinity
for each activity a in sorted list:
    if a.start >= lastFinish:
        add a to selected
        lastFinish = a.finish
return selected
```

2.2 The Three Other Strategies Compared

For each of the three alternative strategies, the scan loop is identical to EFF. Only the sorting criterion changes:

- **Earliest Start First:** sort by s_i ascending — pick activities that begin earliest.
- **Shortest Duration First:** sort by $(f_i - s_i)$ ascending — pick the shortest activities first.
- **Input Order First:** no sorting — scan activities in the original order they were given.

3. Implementation Details

3.1 Data Structures

Each activity is stored in a struct with three fields: name (string), start (int), and finish (int). A helper method `duration()` computes the length of the activity. All activities are kept in a `vector<Activity>`, which is passed by value to each strategy function so the original order is preserved for the next strategy.

3.2 Sorting Method

C++ standard library `std::sort` is used with a lambda comparator. Sorting runs in $O(n \log n)$ time. Because each strategy function receives its own copy of the vector, sorting in one strategy does not affect the others.

3.3 Major Functions

- **`greedyScan(acts)`**: shared scan loop — iterates through the sorted vector, picks compatible activities, tracks `lastFinish`.
- **`earliestFinishFirst`, `earliestStartFirst`, `shortestDurationFirst`, `inputOrderFirst`**: each sorts the vector with a different comparator, then calls `greedyScan`.
- **`readActivities(istream)`**: reads `n`, then `n` lines of name/start/finish from any input stream.
- **`runAllStrategies(acts)`**: prints the input table, then calls and prints all four strategies.

3.4 Compilation and Usage

```
g++ -std=c++17 -o lab5 lab5.cpp
./lab5 sample.txt           # read from file
./lab5 < sample.txt         # pipe from file
./lab5                       # type input manually
```

4. Strategy Comparison

4.1 Sample Input

8 activities: A(1-4), B(3-5), C(0-6), D(5-7), E(3-9), F(5-9), G(6-10), H(8-11)

Strategy	Activities Selected	Count	Optimal?
Earliest Finish First	A, D, H	3	YES
Earliest Start First	C, G	2	No
Shortest Duration First	B, D, H	3	Matches here
Input Order First	A, D, H	3	Matches here

On this input, Earliest Start First clearly fails, it picks C (0-6), which is a long activity that blocks B, D, and H. EFF avoids this by prioritizing activities that free up time quickly.

Shortest Duration and Input Order happen to match EFF on this sample, but that is coincidence, the counterexamples in Section 5 will show they are not reliable.

4.2 Custom Test (self-designed)

6 activities: X(0-3), Y(2-6), Z(3-8), W(6-9), V(8-11), U(9-12)

Strategy	Activities Selected	Count	Optimal?
Earliest Finish First	X, Z, V	3	YES
Earliest Start First	X, Z, V	3	Matches
Shortest Duration First	X, W, U	3	Matches
Input Order First	X, Z, V	3	Matches

All strategies tie on this input because the activities are spread out evenly enough that multiple criteria lead to the same choices. This shows that the alternative strategies are not always wrong — they just cannot be guaranteed to be right.

5. Counterexample Analysis

5.1 Counterexample 1 — Earliest Start First Fails

Input: 3 activities — A(0-10), B(1-2), C(2-3)

Activity	Start	Finish	Duration	Timeline
A	0	10	10 (very long)	0=====10
B	1	2	1	1=2
C	2	3	1	2=3

Earliest Start First result: sorts by start time, picks A(0-10) first. Since A occupies the entire range [0,10], both B and C start inside it and are rejected. Final selection: {A}, count = 1.

Earliest Finish First result: sorts by finish time — B(1-2), C(2-3), A(0-10). Picks B first (finishes at 2), then C is compatible (starts at 2 $\geq$ 2), then A starts at 0 which is less than 3. Final selection: {B, C}, count = 2.

Conclusion: Earliest Start First gets 1 activity while the optimal is 2. Picking the activity that starts earliest blindly can lock us into a long activity that wastes the entire time range.

5.2 Counterexample 2 — Shortest Duration First Fails

Input: 4 activities — A(0-2), B(1-4), C(3-5), D(2-3)

Activity	Start	Finish	Duration	Timeline
A	0	2	2	0==2
B	1	4	3	1===4
C	3	5	2	3==5
D	2	3	1 (shortest)	2=3

Shortest Duration First result: sorts by duration — D(dur 1), A(dur 2), C(dur 2), B(dur 3). Picks D(2-3), then A starts at 0 which is less than 3, skip. C starts at 3 $\geq$ 3, picks C. B starts at 1 < 5, skip. Final selection: {D, C}, count = 2.

Earliest Finish First result: sorts by finish time — A(0-2), D(2-3), B(1-4), C(3-5). Picks A(finish=2), then D(start=2 $\geq$ 2, finish=3), then B(start=1 < 3, skip), then C(start=3 $\geq$ 3). Final selection: {A, D, C}, count = 3.

Conclusion: Shortest Duration gets 2 activities while the optimal is 3. D is the shortest activity but it occupies [2-3], which means if we pick D first, we cannot pick A(0-2) anymore because A comes before D in finish time. Picking D pushed out A unnecessarily.

6. Time Complexity Analysis

6.1 Sorting Phase

All four strategies (except Input Order) use `std::sort`, which has average and worst-case time complexity of $O(n \log n)$, where n is the number of activities. This phase dominates the overall runtime.

6.2 Scanning Phase

After sorting, the greedy scan visits each activity exactly once: one comparison (`a.start >= lastFinish`) and at most one insertion into the result list. This runs in $O(n)$ time.

6.3 Overall Complexity

Phase	Best Case	Average Case	Worst Case
Sorting (<code>std::sort</code>)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Greedy Scan	$O(n)$	$O(n)$	$O(n)$
Total	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Space complexity is $O(n)$ for storing the activities and the result list. No additional data structures are needed.

The Input Order strategy skips sorting, so technically it is $O(n)$ total — but it gives the worst quality results, so the time saving is not worth it.

6.4 Why This Is Efficient

$O(n \log n)$ is essentially the best we can do for this problem, because we need to at least look at all n activities ($O(n)$ lower bound) and any comparison-based sorting algorithm requires $O(n \log n)$ comparisons in the worst case. The greedy approach beats dynamic programming for this unweighted version: DP would take $O(n^2)$ without binary search optimization.

7. Extension: Weighted Activity Selection

In the weighted version, each activity has a value (weight) in addition to its start and finish time. The goal shifts from maximizing the count of activities to maximizing the total value of selected activities.

Greedy no longer works here. For example, with two activities $A(0-10, \text{weight}=100)$ and $B(0-1, \text{weight}=1) + C(1-2, \text{weight}=1)$, greedy would pick $\{B, C\}$ for count=2, but A alone is worth 100 while $B+C = 2$. If we reverse the weights — $A=1, B=C=50$ — greedy picks $\{B, C\}$ correctly, but only by luck.

The correct approach for weighted activity selection is dynamic programming. We sort activities by finish time, then define $dp[i]$ = maximum weight using activities from the first i , and use binary search to find the latest non-conflicting activity for each i . This gives $O(n \log n)$ time but requires more careful bookkeeping than the greedy version.

8. Conclusion and Reflection

This assignment helped me understand that a greedy algorithm is not just about being fast or simple, it has to make choices that are provably optimal, not just intuitively appealing. The three alternative strategies all feel reasonable at first glance: why not start early? Why not pick short activities? But the counterexamples show that local intuition can break down in edge cases.

The key lesson is that Earliest Finish First works because finishing early is the one property that guarantees we never waste more time than necessary. Any activity that finishes later could have been replaced by one that finishes earlier and is at least as compatible with the rest of the schedule.

I also found it interesting that Shortest Duration First can sometimes match the optimal result, which is why naive testing might not catch the bug. A strategy needs to be correct on all inputs, not just most of them. That is why designing specific counterexamples is so important.

If I were to extend this project, I would implement the weighted version using DP and compare it against a greedy attempt on the same inputs to visually show where greedy breaks down. I would also look into interval scheduling on multiple machines (interval partitioning), which is another greedy problem but requires a different criterion.